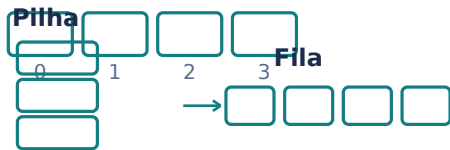


ESTRUTURAS LINEARES



Organizar dados em sequência para acesso, inserção, remoção e processamento.

- **Array:** acesso por índice; tamanho lógico previsível
- **Lista ligada:** nós + ponteiros; inserção/remoção local eficiente
- **Pilha:** LIFO → push, pop, top
- **Fila:** FIFO → enqueue, dequeue, front
- **Deque:** inserção/remoção nas duas extremidades
- **Array:** busca $O(1)$ por índice; inserção interna custa deslocamento
- **Lista:** acesso sequencial $O(n)$; não desloca elementos
- **Escolha:** depende de acesso aleatório vs inserções frequentes

Dica: Se precisa acessar por posição rapidamente, prefira array. Se vai inserir/remover muito no meio, pense em lista ligada.

BUSCA, ORDENAÇÃO & COMPLEXIDADE



Comparar algoritmos pelo custo de tempo e memória para escolher a melhor solução.

- **Big-O:** limite superior assintótico do crescimento
- **Busca linear:** $O(n)$
- **Busca binária:** $O(\log n)$, exige vetor ordenado
- **Bubble/Selection/Insertion:** $O(n^2)$
- **Merge sort:** $O(n \log n)$, estável
- **Quick sort:** médio $O(n \log n)$, pior $O(n^2)$
- **Espaço:** analisar memória além do tempo
- **Trade-off:** melhor caso, pior caso e caso médio importam

Dica: Antes de otimizar código, identifique o gargalo. Trocar $O(n^2)$ por $O(n \log n)$ costuma valer mais que micro-otimizações.

RECURSÃO, ÁRVORES & GRAFOS

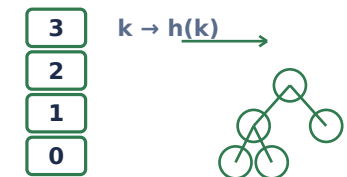


Modelar hierarquias e conexões, explorando problemas por divisão em subproblemas.

- **Recursão:** caso base + chamada recursiva menor
- **Pilha de execução:** cada chamada guarda estado
- **Árvore:** raiz, nós, folhas, altura e subárvores
- **Percursos:** pré-ordem, em-ordem, pós-ordem, largura
- **BST:** esquerda < raiz < direita
- **Grafo:** Vértices + arestas; pode ser dirigido ou não
- **Busca:** DFS usa profundidade; BFS usa fila e níveis
- **Aplicações:** arquivos, redes, caminhos e dependências

Dica: Toda recursão precisa de caso base claro. Para percorrer níveis, BFS costuma ser a escolha natural; para profundidade, DFS.

HASH, HEAPS & BOAS PRÁTICAS



Acelerar consultas e organizar prioridades usando estruturas adequadas ao problema.

- **Tabela hash:** chave → índice por função hash
- **Colisão:** pode usar encadeamento ou endereçamento aberto
- **Custo médio:** busca/inserção/rem. $\approx O(1)$
- **Heap:** árvore quase completa com propriedade de prioridade
- **Max-heap:** pai $\geq$ filhos
- **Min-heap:** pai $\leq$ filhos
- **Fila de prioridade:** inserção e remoção $O(\log n)$
- **ADT:** separar interface da implementação
- **Boas práticas:** modularizar

Dica: Estrutura boa depende da operação dominante: consulta rápida pede hash; retirar maior/menor repetidamente pede heap.